Globally-Accessible Distributed Traffic Service

Dylan Murray (20331999)
Trinity College Dublin
Dublin, Ireland
murraydy@tcd.ie

Raghav Gupta (25360079)
Trinity College Dublin
Dublin, Ireland
guptar3@tcd.ie

Kartik Singhal (25369980)
Trinity College Dublin
Dublin, Ireland
singhaka@tcd.ie

Dylan Thompson (20314016)
Trinity College Dublin
Dublin, Ireland
thompsdy@tcd.ie

Niket Ghai (25361669)
Trinity College Dublin
Dublin, Ireland
ghain@tcd.ie

Stevin Joseph Sebastian (25377614)
Trinity College Dublin
Dublin, Ireland
stevinjs@tcd.ie

1 Introduction

This report presents the design and implementation of a globally-accessible distributed traffic booking service. The system allows drivers to request, confirm and cancel journeys in multiple geographical regions, while supporting validation by enforcement authorities. It uses Apache Kafka for asynchronous message-passing, Redis for low-latency in-memory state management, and MongoDB for durable storage. Our architecture is based on a regionally-partitioned microservices model that exploits data locality to support scalability, availability, and reliability at global scale.

The system addresses key distributed systems requirements including performance, fault tolerance, and durability, while preventing conflicting bookings through distributed locking and maintaining consistency of committed state across regions. It uses distributed techniques such as in-memory caching, geographic data partitioning, saga-based coordination for cross-region journeys, and recovery mechanisms to rebuild Redis states following failure. The system is deployed as a set of loosely-coupled services across three regions deployed using Docker containers, with drivers submitting bookings using a web browser user interface.

2 Requirements

2.1 Functional Requirements

- **Driver Booking Requirements**
  - Drivers can submit bookings with an origin, destination, and departure time.
  - The system returns HTTP 202 immediately with a booking ID. The outcome notification is delivered over WebSocket.
  - Only drivers can create bookings.
  - A vehicle can only hold one active booking at a time. A second attempt is rejected.
  - If the origin or destination can't be found, the system returns HTTP 404.
  - If no road path exists between origin and destination, the system returns HTTP 422.
  - If a vehicle ID is not registered in the system, the booking is rejected.
- **Notification Requirements**
  - After booking, the driver opens a WebSocket connection to receive the result.
  - The server pushes APPROVED or REJECTED as a JSON message once validation completes.
  - For cross-region bookings, only the final outcome is sent to the driver.
  - If the WebSocket fails, the driver can view their booking status using a REST endpoint.
- **Booking Lifecycle Requirements**
  - Drivers can view their full booking history, including status (approved, pending, cancelled, rejected).
  - Drivers can cancel an active booking, only if the booking belongs to them.
  - On cancellation, the road segment counters are decremented so capacity is freed up.
  - Drivers can check the state of a cross-region booking.
- **Authority Requirements**
  - All authority endpoints require the AUTHORITY role.
  - Authorities can search a vehicle ID, and the system returns all bookings for that vehicle.
  - Authorities can view the most recent bookings logs across all drivers in the region.
  - Authorities can flag a booking, the flag is written to MongoDB.
- **Validation Requirements**
  - Each regional Validation Service only processes bookings tagged for its own region.
  - For each booking it: acquires Redis locks on all route segments, checks capacity, checks for a duplicate vehicle booking, increments counters, and publishes the result.
  - For cross-region bookings, the final booking record is only written to MongoDB after all regions approve.
  - The Validation Service also handles cancellations by decrementing segment counters.
- **Routing Requirements**
  - The system computes a road-network path between any two locations.
  - Cross-region journeys are joined together at border gateway nodes automatically.
  - If no path exists within a region, the system returns HTTP 404.
- **Account Requirements**
  - Users register with a username, password, and vehicle ID. Role is selected at registration.
- **Observability Requirements**
  - Every service exposes a `/health` endpoint.

  - A health dashboard polls all services and shows each services status.

2.2 Non-Functional Requirements

- **Performance**
  - The system should respond to booking submissions with low latency. We achieve this by returning HTTP 202 immediately after publishing to Kafka (measured p99: 220 ms at normal load).
  - The system should complete cross-region bookings quickly. We achieve this by fanning out sub-bookings to all regions in parallel rather than sequentially (measured mean: 385 ms).
  - The system should perform capacity checks without bottlenecking. We achieve this by keeping all counters and locks in Redis memory.
  - The system should compute routes efficiently. We achieve this using A* Search Algorithm.
- **Scalability**
  - All services are stateless and can be horizontally scaled by adding Docker replicas behind NGINX with no configuration changes.
  - The system should distribute validation load across replicas. We achieve this using Kafka consumer groups, when Validation Service replicas are added Kafka automatically balances partitions.
  - The system is geographically partitioned by region. Each cluster operates independently and new regions can be added by importing OSM data with no code changes.
  - Road segment capacity limits are stored in Redis and can be updated live without restarting any service.
- **Availability**
  - If a regional cluster goes down, DNS stops routing to it and redirects drivers to a healthy region automatically.
  - If a container crashes, Docker restarts it automatically using the `restart: unless-stopped` policy.
  - If the local cluster is unreachable, the Data Gateway falls back to MongoDB to continue processing read requests.
  - The system tolerates the loss of one or two regions. Loss of all three simultaneously is not handled.
- **Reliability**
  - Kafka consumers replay unprocessed messages on restart so no booking is lost.
  - MongoDB operations are retried up to 3 times with exponential back off before returning HTTP 503.
  - Redis lock TTL of 10 s releases orphaned locks if the Validation Service crashes mid-sequence.
  - Saga documents are written to MongoDB before sub-bookings are published, so the coordinator can recover on restart.
- **Data Management**
  - Redis: holds segment capacity limits, running counters, distributed locks, and active vehicle bookings. Keys are rebuilt from MongoDB after a restart.
  - MongoDB: stores bookings, sagas, users, flags, and audit logs. After a booking is approved, the record is immediately available to query with no eventual-consistency lag. MongoDB Atlas replicates data across regions with automatic failover.
  - Kafka: messages are retained for a set amount of time before auto-deletion.

3 Specification

3.1 System Overview

The pre-booking journey functionality system is made up of multiple distributed microservices deployed on Docker. Drivers connect through an NGINX API gateway, which proxies requests to backend services. Booking validation is fully asynchronous: the Booking Service publishes to Kafka and returns 202 Accepted immediately. The driver receives the final outcome from a WebSocket once the regional Validation Service finishes its checks. GeoDNS automatically routes each driver's request to the nearest healthy regional cluster. If two drivers book the same road segment at the same time, the distributed locking protocol guarantees only one is approved. Bookings submitted to a healthy region are processed normally even if another region is completely unreachable. Multiple active bookings are rejected.

3.2 Components

Description of the key components:

- **User Client:** A web application served by NGINX. Provides driver registration, login, booking submission, booking history, cancellation, and real-time notifications. Also includes an authority panel and a health dashboard.
- **GeoDNS:** A custom DNS resolver that maps incoming client requests to the nearest regional cluster based on their IP prefix. It runs health checks and removes unhealthy regions from rotation automatically.
- **NGINX:** Acts as the API gateway and load balancer for each regional cluster. Proxies requests to backend services and serves the static frontend.
- **Booking Service:** Validates the driver JWT and calls Journey Management to resolve the route. Publishes bookings to Kafka for single-region journeys or the saga coordinator for cross-region ones.
- **Journey Management Service:** Resolves the full route across regions and acts as the saga coordinator for cross-region bookings.
- **Route Service:** Loads the OSM road graph for its region into memory at startup and uses A* shortest-path algorithm.
- **Validation Service:** Kafka consumer that acquires Redis locks on route segments, checks capacity, increments counters, and publishes outcomes.
- **Data Service:** Data_Service is its own microservice running on its own container. All MongoDB reads and writes from application services go through this data layer, which handles retries, connection management, and hash sharding of road segment data. Redis is accessed directly by the Validation Service for low-latency locks, but the Data Service provides the rebuild path. `rebuild_redis.py` reads confirmed bookings from MongoDB through the Data Service and reconstructs Redis counters after a restart.

- **Data Gateway:** Routes requests to the correct regional Data Service.
- **Authority Service:** Allows vehicle lookup, log search, and booking flag submission. Accepts only AUTHORITY role JWTs.
- **Notification Service:** Consumes `booking-outcomes` from Kafka and pushes results over WebSocket to the driver.
- **User Registry:** Handles user registration.
- **Kafka:** Handles async messaging between services. Ensures bookings are not lost if a service restarts.
- **Redis:** Stores hot data, capacity counters and locks. Accessed directly by the Validation Service for speed. Lock TTL is 10 s, which releases orphaned locks if a process crashes. State is rebuilt from MongoDB after a restart.
- **MongoDB:** Stores all data including bookings, users, sagas, flags, audit logs, and OSM road graph data. All application writes go through the Data Service layer. Source of truth.

3.3 Fault Tolerance and Error Handling

- If a Validation Service crashes mid-validation, Kafka redelivers the message to the restarted container. A duplicate vehicle check prevents double-approval.
- If MongoDB is temporarily unavailable, the Data Service retries a preset amount of times. After exhaustion it returns 503.
- Each region has its own local MongoDB. If it is down, the Data Gateway falls back to global MongoDB Atlas for reads.
- If Redis fails/restarts, `rebuild_redis.py` reads capacity counters from MongoDB. New bookings queue in Kafka and are processed once counters are restored.
- If a regional cluster is unreachable, the Data Gateway falls back to MongoDB Atlas for read requests. GeoDNS stops routing new bookings to the failed region. The system tolerates the simultaneous failure of one or two regions and the remaining region(s) continue accepting and processing bookings normally. Simultaneous failure of all three regions is not tolerated.
- If the saga coordinator crashes, it reconstructs state from the global MongoDB sagas collection and Kafka's uncommitted consumer group offsets, which redeliver any unprocessed sub-booking outcomes to the new leader instance.
- If the WebSocket connection drops after booking submission, the driver can use `GET /bookings/{id}` to see the outcome.
- Kafka doesn't commit until outcome is published so if fails before approval nothing is lost.
- Restart: unless-stopped on every Docker container.

3.4 Limitations

Some features are not fully implemented or are limited in scope:

- The Docker Compose stack runs a single Kafka broker, which is a single point of failure. A multi-broker setup was not implemented.
- OSM road data is only imported for three small regions: Andorra, Laos, and Cambodia.
- The local MongoDB container per region is a single point of failure. The Data Gateway falls back to Atlas for reads only not writes. A MongoDB replica secondary would fix primary failures in a real production system.
- Redis is not replicated. On restart, the Validation Service watchdog automatically re-seeds capacity limits from MongoDB within roughly thirty seconds. Live occupancy counters reset to zero during this window, which is a known accuracy tradeoff rather than a hard outage.

3.5 Distributed Techniques

Process Replication: Stateless services (Booking, Validation, Route, Notification) are containerised. Multiple replicas can run behind NGINX with no configuration changes.

- *Degree of Isolation:* Each replica is independent. A single container crash does not affect others.
- *Degree of Replication:* As many replicas as needed. Kafka consumer groups distribute messages across Validation Service replicas automatically.
- *Consistency:* Stateless replicas share no local state. All shared state in Redis or MongoDB.
- *Fault Tolerance:* Container failures are detected and restarted by Docker within seconds.

Partitioning: Road network data and validation state are split by geographic region. Each region runs its own Route Service, Validation Service, Redis, and MongoDB. No cross-region data access is required for single-region bookings. During a network partition, each region continues operating independently on its own road data and Redis state. Bookings submitted to a reachable region are processed normally even if other regions are unreachable.

A failure in one region does not block another. Each region's Redis and MongoDB hold only their own data. Road graph data is read-only at runtime. Write contention is localised to each region's Redis instance. A regional failure causes DNS to redirect traffic. Other regions continue normally. Network partitions between regions are tolerated. Each region continues independently.

Caching: Redis stores all real-time validation state: capacity counters, distributed locks, and active vehicle bookings, in memory. This removes MongoDB from the hot path entirely. Each region runs its own Redis instance, so caching is sharded by region. No cross-region Redis communication occurs.

- *In-memory?:* Yes. All lock capacity limits, and counters live in Redis for fast access.
- *Replacement strategy:* Lock keys expire after 10 s. Capacity is rebuilt from MongoDB by `rebuild_redis.py` after a restart.
- *Degree of Isolation:* Redis holds rebuildable derived states. MongoDB is the authoritative store. A Redis failure stops new validations but does not change booking records.
- *Fault Tolerance:* Redis restart triggers new bookings queue in Kafka and are processed once counters are restored.
- *Crash Stop Failure:* Counters are verified against MongoDB confirmed bookings during rebuild.
- *Update strategy:* Write-through. Every approved booking is written to MongoDB before the APPROVED outcome is published. Redis counters are incremented only after the MongoDB write succeeds, so Redis is never ahead of the durable store.

Database Partitioning (Hash Sharding): When querying the database, services contact the Data Service which provides APIs to abstract all database access, so that services don't need to worry about data formatting or where data is stored. The Data Service applies a hash function to decide which shard to query. The formula is:

$$\text{shard_id} = \text{road_id} \bmod \text{total_shards}$$

This returns a value in the range $[0, \text{total_shards}-1]$, which maps to a shard server ID (1, 2, 3, ..., n). For example, to query the capacity of road segment 400 with 5 shards, the Data Service computes $400 \bmod 5 = 0$, and forwards the query to shard server 0.

Distributed Locking: Redis prevents double-booking on shared road segments using locks. Locks are per road segment. Contention on one segment does not block others. This also works for cross-region booking being isolated on each region.

Load Balancing: NGINX distributes client requests across service replicas within a region. DNS routes requests to the nearest healthy regional cluster based on client IP. NGINX and the DNS resolver are stateless and can be replaced without interruption. At least one regional cluster is always reachable.

Transactions: Cross-region bookings use the Saga pattern. The coordinator fans out sub-bookings to each region using Kafka and either commits or rolls back all regions. The saga document is written to MongoDB first so the coordinator can recover on restart. This is preferred over 2PC as a crashed 2PC coordinator would block all regions until recovery.

Replication: MongoDB Atlas replicates the `traffic` database across regions. If the local cluster is unreachable, the Data Gateway falls back to Atlas for reads only. Local MongoDB container failures do not cause data loss.

Message Logging: Each service logs errors and key events to stdout. The Data Service uses Python's `logging` module to log MongoDB retry warnings and failures. The Validation Service logs consumer errors and Redis seed results on startup. The Notification Service logs failed WebSocket broadcasts. Journey Management logs saga advance errors. GeoDNS logs every DNS query, region selection, and unresolvable request. Authority flags and booking actions are written to a persistent logs collection in MongoDB.

Segment-Level Locking and Isolation: The system uses segment-level locking to enforce serialisable isolation, preventing dirty reads and phantom bookings by requiring transactions to acquire locks in a fixed sequence before modifying any segment counters.

Majority Partitions: No quorum is enforced. A minority partition continues operating independently. On reconnect, Atlas replica set reconciliation resolves divergence. Loss of all three regions simultaneously is not tolerated.

3.6 API Descriptions

- **FastAPI (REST):** The Booking, Journey Management, Route, Authority, User Registry, Data Service, and Data Gateway services expose HTTP endpoints built with FastAPI. Handles booking submission, cancellation, booking history, authentication, authority lookups, and health checks.
- **WebSocket API:** The Notification Service pushes booking outcomes (APPROVED or REJECTED) to drivers in real time. The driver connects after submitting a booking and receives the result once the Kafka pipeline completes.
- **Kafka API:** Services communicate asynchronously using Kafka. The Booking Service publishes new bookings, Validation Services consume and publish outcomes, and capacity releases are sent on cancellation or abort. Messages are not lost if a service restarts.
- **Redis API:** The Validation Service accesses Redis directly for distributed locks, capacity counters (`INCR`/`DECR`), and active vehicle flags. Caches the validation hot path in memory without going through the Data Service.
- **MongoDB API:** All persistent reads and writes go through the Data Service. Stores bookings, users, sagas, flags, audit logs, and OSM road graph data.
- **DNS API:** GeoDNS responds to DNS queries by returning the IP of the nearest healthy regional cluster. Runs health checks and removes unhealthy regions from rotation without any client-side changes.
- **Nominatim API:** The Route Service and Journey Management call the OpenStreetMap Nominatim geocoding API to convert place names to coordinates. The result is snapped to the nearest OSM road node for routing.

4 Architecture

4.1 Overview

The system follows a distributed microservices architecture that separates client-side interaction, service-side logic, and backend storage. It consists of a client, backend services, and supporting infrastructure including Redis, Kafka, and MongoDB. Clients connect through GeoDNS, which resolves requests to the nearest healthy regional cluster. Each cluster is fronted by an NGINX gateway, which serves the frontend and proxies API requests to backend services. The system is deployed across three geographic regions enabling fault isolation and geographic partitioning.

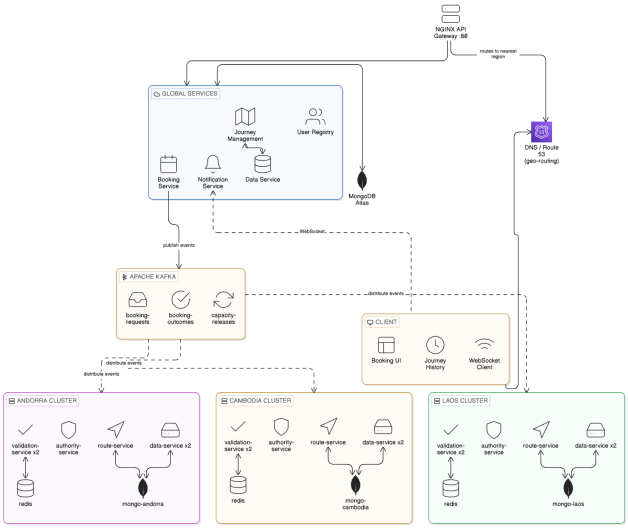

Figure 1: System architecture showing global services, regional clusters, and communication paths.

4.2 Technical Architecture

At the start of the system, the driver interacts with the frontend UI through a web browser. GeoDNS resolves requests to the nearest healthy deployment, providing simple regional failover. NGINX then serves the static frontend and proxies API calls to backend services.

The main control-path services are the Booking Service, Journey Management Service, Notification Service, User Registry, and a global data layer for shared records such as sagas, users, flags, and audit logs. The Booking Service is the entry point for journey requests. Journey Management performs route planning and also coordinates cross-region saga execution. Notification Service delivers final outcomes over WebSocket. User Registry handles registration and login.

Each regional cluster contains the services needed to validate and store journeys locally. Route Service holds the regional OSM graph in memory and computes shortest paths using A*. Validation Service consumes booking requests from Kafka, acquires Redis locks, checks capacity, prevents duplicate vehicle bookings, and publishes outcomes. Data Service handles persistent reads and writes for that region. Redis stores hot in-memory state such as locks, capacity counters, and active vehicle flags, while MongoDB stores durable booking and graph data.

Kafka connects the services asynchronously. Booking requests, validation outcomes, and capacity release events are published to Kafka topics, which decouples submission from processing and allows retries and replay after crashes. MongoDB Atlas provides replicated global persistence and read fallback when a regional database is unavailable.

4.3 Runtime Services and Distribution

The runtime services are distributed across a global layer and three regional clusters.

- **Global layer:** Booking Service, Journey Management Service, Notification Service, User Registry, Kafka, and MongoDB Atlas.
- **Regional layer:** Route Service, Validation Service, Data Service, Authority Service, Redis, and MongoDB for Laos, Cambodia, and Andorra.
- **Ingress layer:** GeoDNS and NGINX provide request routing, failover, and API gateway behaviour.

This distribution keeps validation close to regional road data while allowing cross-region bookings to be coordinated centrally. Single-region bookings stay mostly within one cluster. Cross-region bookings use the global coordinator and Kafka to fan out validation work to the regions involved.

4.4 Component Connectivity

The main service connections are as follows:

- Driver → GeoDNS → NGINX gateway
- NGINX → Booking Service / User Registry / Authority endpoints / frontend assets
- Booking Service → Journey Management for route planning
- Booking Service → Kafka for booking publication
- Journey Management → Route Services for regional path computation
- Journey Management → Kafka for saga fan-out and coordination
- Validation Services ← Kafka booking topics
- Validation Services ↔ Redis for locks, counters, and active vehicle checks
- Data Services ↔ MongoDB for durable booking and graph data
- Notification Service ← Kafka outcomes topic
- Notification Service → driver WebSocket connection
- Data Gateway → regional Data Services, with fallback to MongoDB Atlas for reads

4.5 Architectural Design Rationale

The architecture was designed around four main goals: low-latency validation, fault isolation, scalability, and safe cross-region coordination.

- **Low latency:** Redis keeps locks and counters in memory so validation does not depend on slow database round-trips.
- **Fault isolation:** Each region owns its own route graph, validation state, and local database, so a regional failure does not stop the whole system.
- **Scalability:** Stateless services can be replicated horizontally, and Kafka consumer groups distribute validation load automatically.
- **Cross-region correctness:** Saga coordination ensures that a journey touching multiple regions either commits fully or rolls back cleanly.

4.6 Individual Service Design

4.6.1 Journey Management Service, Route Service — Kartik Singhal.

Description: The Journey Management Service acts as the global route planner and cross-region saga coordinator. It holds a coarse

region-level overlay graph and uses Dijkstra's algorithm to determine which regions are involved in a journey, then delegates each leg to the appropriate regional Route Service via HTTP. Each Route Service loads its region's full road graph from MongoDB at startup and serves A* shortest-path queries using a haversine heuristic. For cross-region journeys, Journey Management fans out one sub-booking per region to Kafka, collects regional validation outcomes, and either commits the booking to every involved region's data store or publishes compensating capacity-release messages to roll back approved regions. Two instances run concurrently, with Redis `SET NX` leader election ensuring only one instance drives the saga coordinator Kafka consumer at a time.

Failure handling:
If the leader instance crashes, its Redis lock key expires after its 30-second TTL and the standby instance wins the next `SET NX` attempt, resuming saga coordination within 30 seconds. Kafka retains all unconsumed outcome messages during this window, so no saga decisions are lost. All HTTP calls from Journey Management to the regional Route Services use `tenacity` retry logic with three attempts and exponential backoff, which masks brief route-service restarts transparently. The Route Service loads its road graph once at startup into memory, so a post-startup MongoDB outage does not interrupt routing. A known limitation is that sagas in flight at the exact moment of a leader crash remain in `PENDING` state, as no there is currently no saga-timeout watchdog to trigger recovery.

4.6.2 *Booking Service — Stevin Joseph Sebastian.*

Description: The Booking Service is the only driver-facing entry point for all journey submissions in our system. For each booking request it receives, it does the following: Validate the incoming JWT, enforce the `DRIVER` role, and call Journey Management to plan the route. **Single-region journeys:** the service publishes directly to the Kafka `booking-requests` topic. **Cross-region journeys:** It delegates to Journey Management's saga endpoint.

A module-level Kafka producer is shared across all request threads via a `threading.Lock`. This is done to avoid per-call connection overhead that can result in severe queueing latency under high concurrency. The service maintains an in-memory outbox that works in the following manner: if the Kafka broker is temporarily unreachable, the message is queued and a background thread retries delivery every five seconds. The service also implements an idempotency cache that is keyed on the `X-Idempotency-Key` header (300-second TTL). This is to ensure that client retries do not produce duplicate bookings.

Failure handling:
Two instances run behind an nginx upstream with `least_conn` load balancing and `proxy_next_upstream` failover. This is to ensure that a single instance crash is invisible to clients.

The singleton Kafka producer eliminates the 24-second median submission latency observed under 1000-way concurrency with the previous per-call producer design.

The outbox decouples the driver-facing 202 response from Kafka availability. The driver receives confirmation immediately, and delivery is guaranteed once the broker recovers.

All HTTP calls to Journey Management use `tenacity` with three retries and exponential backoff. This is to prevent transient planning failures from showing up as errors to the client.

4.6.3 *NGINX, Docker Clusters, and Authority Service — Dylan Murray.*

Description: When deciding how to handle geo partitioning requests, we decided that it would be best to have a server cluster for each region. We build a DNS that holds the IP Address for the nginx gateway to the region. The client will contact the DNS with their location, and it will respond with the address that they should connect to. This can be expanded to have multiple clusters per region and have land balancing across them. the client will then begin sending their http requests to the address they have received. This address is for a nginx node that sits at the edge of our cluster. The nginx is configured to forward requests across the multiple instances of our booking service and will also load balance. The nginx will also respond to a "/health" request so that the client can ensure that the service is up. Nginx was decided as it is the industry standard, I had experience with it in the past, and it covered all of the requirements. The DNS was built in house we could make it more lightweight than what is currently offered, and tailer it closer to the needs that we already face and might appear.

4.6.4 *Validation Service — Raghav Gupta.*

Description: The Validation Service acts as the system's capacity enforcement layer. It operates on two independent Kafka consumer loops: one that processes incoming booking requests from the `booking-requests` topic, and another that handles compensating rollbacks from the `capacity-releases` topic. For each booking request, the service acquires a Redis lock (`SET NX`) for every road segment, ordering them deterministically to avoid deadlocks. It then verifies that the current occupancy counter does not exceed the predefined capacity and checks with the regional data service to ensure the vehicle does not already have an active booking.

If all validations pass, the service atomically increments the Redis occupancy counter, persists the booking in MongoDB, and publishes an `APPROVED` outcome to Kafka. If any validation step fails, it publishes a `REJECTED` outcome without modifying the occupancy counters. Capacity limits are preloaded into Redis at startup from the `osm_edges` collection, with values ranging from 200 vehicles for motorways to 1 vehicle for single-track roads.

Failure Handling: Each Kafka consumer loop runs within a `while True` block with exception handling and a five-second retry delay. This ensures that a failure in one loop does not impact the other and allows automatic recovery without requiring a container restart. To handle Kafka's at-least-once delivery semantics, the service first checks the booking's status in MongoDB before processing; if the booking already has a terminal state, the message is treated as a duplicate and ignored.

A Lua script (`_safe_decr`) is used for all capacity-release operations to ensure that occupancy counters never drop below zero, even if release messages are processed multiple times. If the MongoDB

write fails after capacity has been reserved, the service publishes a `REJECTED` outcome and avoids incrementing the Redis counter, ensuring that no booking is considered valid unless it is successfully persisted in the database.

4.6.5 *Notification Service User Registry - Niket Ghai.*

Description: The Notification Service sends booking results to drivers via persistent WebSocket connections. Its Kafka consumer ignores sub-booking messages (`is_sub_booking: true`) and forwards only final outcomes by publishing them to a Redis pub/sub notifications channel. Each service instance runs a loop that listens to this channel and delivers messages to any WebSocket connections it maintains for the relevant driver. This design allows multiple instances to share delivery responsibility without requiring coordination.

To safely handle concurrent WebSocket activity, the connections dictionary is protected using an `asyncio.Lock`, preventing race conditions.

The User Registry acts as the central identity service. It generates JWTs with roles such as `DRIVER` or `AUTHORITY`, secures passwords using bcrypt hashing, and enforces unique usernames through a MongoDB index created at startup.

Failure Handling: If a Notification Service instance crashes, its WebSocket connections are lost and clients reconnect via nginx, which automatically routes traffic away from the failed instance using `ip_hash`. The remaining instance continues to receive notifications from Redis and delivers them to reconnected drivers.

If a driver is disconnected when outcomes are published to Kafka, any active Kafka consumer processes the messages and forwards them to Redis; the driver can also retrieve results through the `GET /bookings/id` endpoint.

In the User Registry, duplicate registration attempts raise a MongoDB `DuplicateKeyError`, which is handled by returning a `409 Conflict` response instead of a server error. Additionally, stale WebSocket connections in the Notification Service are automatically cleaned up when message delivery fails.

4.6.6 *Data Service — Dylan Thompson.*

The Data Service acts as the internal data layer, handling all MongoDB interactions and coordinating access across the system. All reads and writes from the system go through this service, which handles database connections and includes retry logic to deal with temporary failures. MongoDB is used for durable storage, with each regional cluster running its own instance for low-latency access, while Redis is used for faster operations like distributed locking and caching hot data such as capacity. Redis is accessed directly by the Validation Service to keep latency low. It is deployed as two instances per region to eliminate it as a single point of failure.

To handle failures, the Data Service supports rebuilding Redis state from MongoDB. `rebuild_redis.py` scans confirmed bookings and rebuilds road capacity counters and active vehicle states after a Redis restart. This design works well in a distributed system because it clearly separates long-term persistent data (MongoDB) from fast, in-memory state (Redis), meaning Redis can fail and be rebuilt without losing any critical data, while still allowing the system to scale and perform efficiently. If both instances of a regional Data Service become unavailable, the Validation Service publishes a `REJECTED` outcome rather than leaving a booking in a partially committed state. This prevents any inconsistency between Redis counters and the booking record.

5 Implementation

Overall Operation:

- When a driver submits a booking through the web app, the system first validates their JWT token and assigns a unique booking ID before doing anything else.
- Journey Management is then responsible for turning the origin and destination into actual road segments, computing the full route using the regional graph data.
- If the route stays within a single region, the Booking Service drops the request straight onto Kafka and immediately hands the driver back an HTTP 202 so they are never left waiting.
- When the route crosses regional boundaries, Journey Management takes over coordination by writing a saga record to MongoDB and sending one sub-booking per region onto Kafka simultaneously.
- Each region runs its own Validation Service, which picks up the sub-booking, locks all the relevant road segments in a consistent order using Redis, checks whether capacity is still available, confirms the vehicle has no conflicting active booking, and then publishes its decision.
- The saga coordinator inside Journey Management waits for every region to respond. A full set of approvals triggers a confirmed booking write to MongoDB and a COMMITTED message on Kafka. A single rejection causes it to undo any capacity already reserved in other regions and publish ABORTED instead.
- Finally, the Notification Service reads the outcome off Kafka and delivers it to the driver in real time over their open WebSocket connection.

5.1 Flow Diagrams

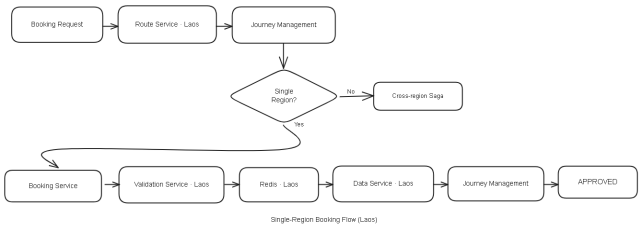

Figure 2: Single-Region Booking Flow (Laos)

The booking request is routed through the Laos Route Service and Journey Management, then published to the Validation Service, which acquires Redis locks, increments the capacity counter, and persists the record before returning an approved outcome.

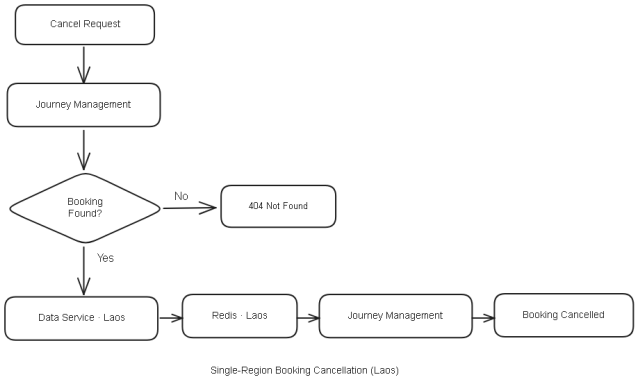

Figure 3: Single-Region Booking Cancellation (Laos)

A cancellation request is handled by Journey Management, which looks up the booking and, if found, marks it cancelled in the regional Data Service and releases the reserved capacity back to Redis.

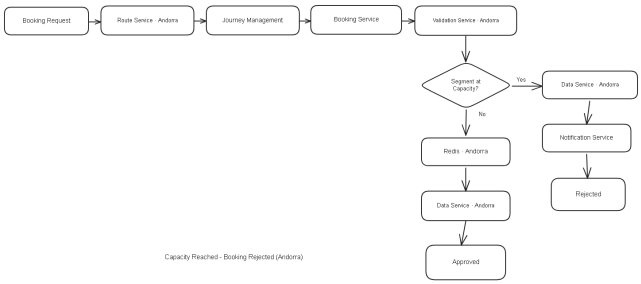

Figure 4: Capacity Reached Booking Rejected (Andorra)

When the Andorra Validation Service finds a segment already at capacity, it writes the rejected booking to the Data Service and notifies the driver using the Notification Service

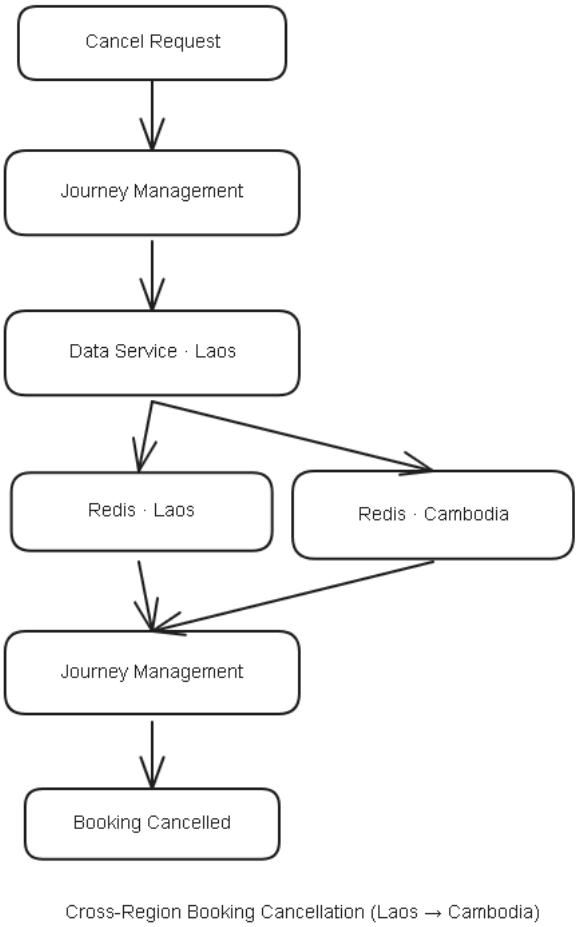

Figure 5: Cross-Region Booking Cancellation (Laos to Cambodia)

Cancelling a cross-region booking triggers Journey Management to publish capacity-release messages to every involved region. Here they decrement the Redis counters in both Laos and Cambodia before confirming the cancellation.

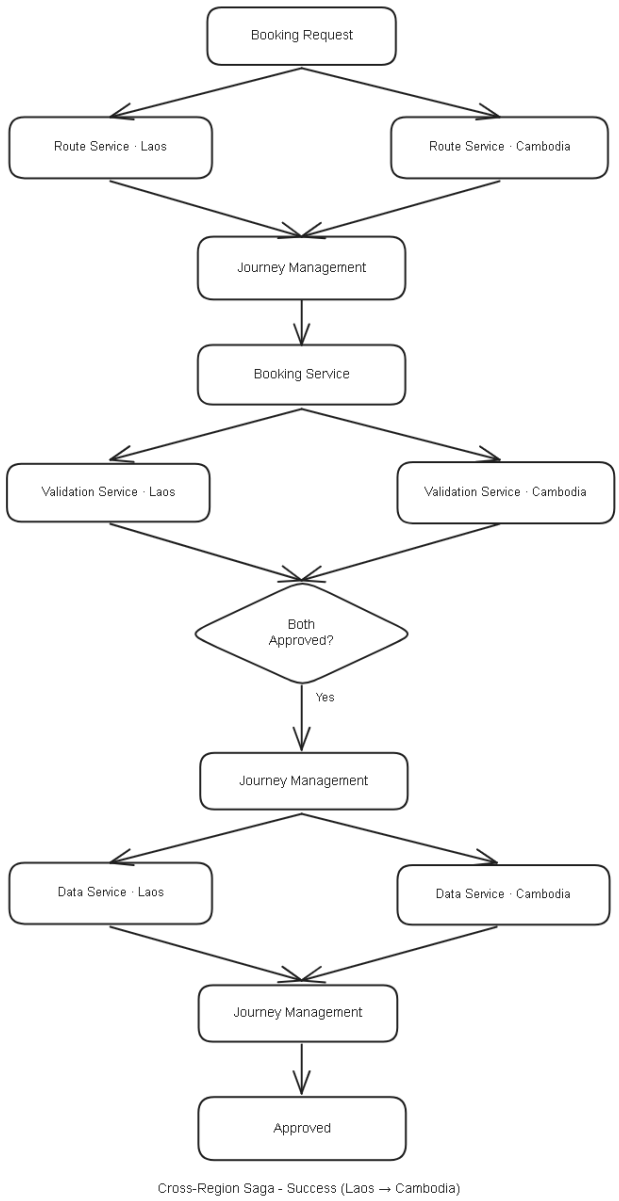

Figure 6: Cross-Region Saga Success (Laos to Cambodia)

Both regional Validation Services independently approve their sub-bookings. Journey Management collects both outcomes, commits the booking record to each region's Data Service, and returns an approved outcome.

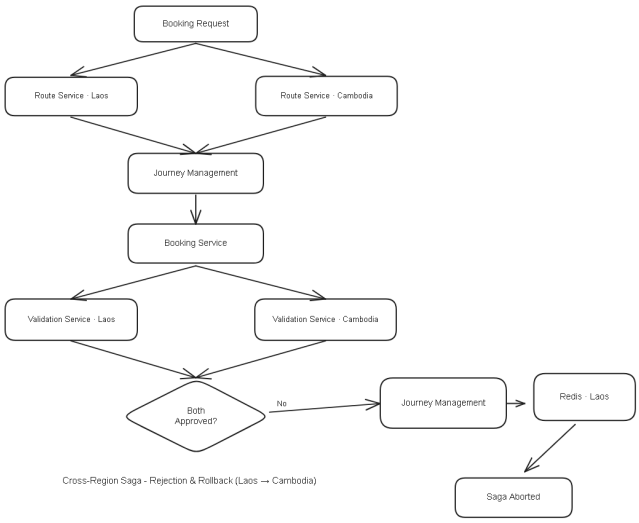

Figure 7: Cross-Region Saga Rejection and Rollback (Laos to Cambodia)

When at least one regional validation rejects its sub-booking, Journey Management aborts the saga, publishes a capacity-release message to roll back any already-approved regions, and returns a rejected outcome to the driver.

5.2 Failure Handling

- **Container crash:** Docker brings the container back up on its own. Because Kafka holds onto any message that was not fully acknowledged, it gets redelivered to the restarted service. A duplicate vehicle check at the start of validation makes sure the same booking cannot be approved a second time.
- **Kafka broker failure:** Once the broker restarts, every consumer reconnects automatically and resumes from its last committed offset. The duplicate vehicle check ensures that any redelivered message does not get processed again. While it is down, Booking Service queues the outgoing messages in its own in-memory outbox and retries every 5 seconds. Submissions accepted during outages are not lost.
- **Redis restart:** A restart wipes all the in-memory counters and locks. The Validation Service watchdog detects the empty state and automatically re-seeds capacity limits from the `osm_edges` collection in MongoDB in thrity seconds. A known limitation is that live counters are reset to zero on recovery, but the system is safe against deadlock.
- **MongoDB unavailable:** If a write to MongoDB fails, the Validation Service does not retry the booking. Instead it immediately publishes a `REJECTED` outcome to Kafka. This ensures that no booking can exist in an approved state without a database record. The connection pool has a fie second server selection timeout.
- **Regional cluster goes down:** GeoDNS detects the unhealthy region within roughly a minute and stops sending drivers there. Any read queries that would have gone to the

regional MongoDB instance are handled by MongoDB Atlas as a fallback.

- **Saga coordinator crashes:** Because the saga document is written to MongoDB before any sub-bookings go out on Kafka, nothing is lost. When the coordinator comes back up it reads the document, sees exactly which outcomes have already arrived, and carries on from that point. However, sagas that the crashed leader was actively processing in memory are not recovered — they remain in `PENDING` state indefinitely, as the new leader does not scan for and resume in-flight sagas on startup. This is a known limitation.

5.3 Tools and Libraries

- **FastAPI + uvicorn:** Powers all nine backend services. Both regular HTTP endpoints and WebSocket connections run asynchronously through the same stack.
- **Apache Kafka (python-kafka):** Acts as the messaging backbone between services. If a service goes down while processing a message, Kafka holds it until the service is ready to try again.
- **Redis (redis-py):** Keeps road capacity counters and segment locks in memory so every check is fast and results stay consistent across concurrent requests.
- **MongoDB (PyMongo):** The single source of truth for everything that needs to survive a restart: bookings, users, sagas, and the road graph itself.
- **httpx:** Handles all the async HTTP calls between services. Each call carries a timeout so one slow service cannot stall the entire request chain.
- **tenacity:** Wraps MongoDB operations in automatic retry logic, trying up to three times before accepting that the operation has failed.
- **geopy / Nominatim:** Turns the place names a driver types in into coordinates, which are then snapped to the nearest node in the road graph.

6 Testing

6.1 Test Plan

Testing was designed to validate both the functional correctness and non-functional behaviour of the distributed traffic booking system.

- **Functional correctness:** Ensuring all system features behave as expected, including booking creation, cancellation, cross-region journeys, and authority operations.
- **Performance:** Measuring request latency under normal operating conditions.
- **Scalability:** Evaluating system behaviour under scaling load.
- **Fault tolerance:** Verifying recovery from service failures such as container crashes and network partitions.

All tests were executed on the full Docker Compose deployment using `scripts/load_test.py`. A custom road graph was used to eliminate external geocoding delays, ensuring that results reflect only internal system performance.

6.2 Functional Requirements Testing

The system was tested end-to-end to ensure all functional requirements were met.

- **System deployment:** The Docker Compose setup was tested to ensure all services start correctly with a single command. All containers (Kafka, Redis, MongoDB, and services) were successfully initialised and reachable via their health endpoints.
- **Booking submission:** Drivers were able to submit bookings with valid JWTs. The system returned HTTP 202 immediately, and booking outcomes were delivered asynchronously.
- **Single-region booking:** Bookings within a single region were tested for both approval and rejection cases. When capacity was available, bookings were approved, when a segment reached capacity, bookings were correctly rejected.
- **Cross-region booking:** Journeys from Laos to Cambodia were tested. The saga coordinator correctly fanned out sub-bookings and returned a single final outcome (COMMITTED or ABORTED) based on regional capacity.
- **Booking cancellation:** Drivers were able to cancel active bookings. Capacity counters were decremented correctly, making road segments available again.
- **Booking history:** Drivers could retrieve all previous bookings with accurate status values (PENDING, APPROVED, CANCELLED, REJECTED).
- **Authority operations:** Authority users successfully queried vehicle booking history and submitted flags with reasons stored in MongoDB.
- **Real-time notifications:** WebSocket connections delivered booking outcomes to drivers immediately after validation completed.

Additional unit tests were implemented for the Validation Service using mocked Redis and MongoDB instances. These tests verified:

- Correct lock acquisition and release behaviour
- Capacity enforcement logic
- Duplicate vehicle booking detection
- Correct rollback behaviour during saga abort
- Redis restart correct rebuild from MongoDB

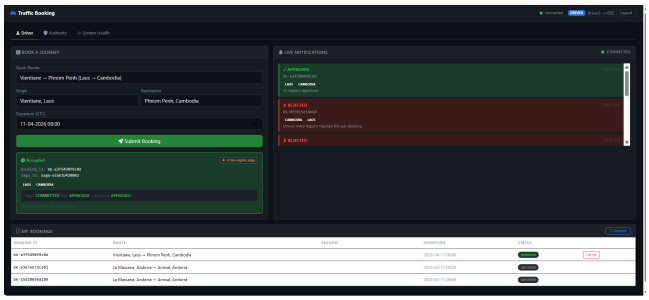

Figure 8: Driver interface showing booking submission, real-time WebSocket notifications, and booking history. The example demonstrates a successful cross-region saga (Laos to Cambodia) and rejected bookings.

Load levels were chosen to reflect expected usage patterns: 5 req/s models a steady regional booking rate during normal hours, and 50 concurrent requests simulates peak-hour contention when multiple drivers attempt to book overlapping road segments simultaneously.

6.3 Non-Functional Requirements Testing Through Load Simulation

Non-functional testing evaluated system performance, scalability, reliability, and fault tolerance under realistic and stress conditions. All tests were executed against the full Docker Compose deployment, with all services active across three geographic regions. Performance measurements were taken using a custom load testing script, while correctness and distributed behaviour were validated using logs.

6.3.1 Performance Testing. **Baseline Performance.** A baseline was taken using 100 sequential requests to the `/api/health` endpoint to establish the minimum system overhead.

- p50 latency: 6 ms
- p95 latency: 9 ms
- Maximum latency: 14 ms
- Success rate: 100%

Captures only NGINX proxying and FastAPI response time without application logic.

Single-Region Booking (Sustained Load). Approximately 2,900 booking requests were issued at a target rate of 50 requests per second over 60 seconds.

- Achieved rate: 48.1 req/s (96% of target)
- Success rate: 100%
- p50 latency: 81 ms
- p95 latency: 792 ms
- p99 latency: 1,189 ms
- Mean latency: 177 ms
- Maximum latency: 1,532 ms

Under a steady paced load the system achieves near-baseline p50 latency. The p95 tail reflects occasional bursts when multiple requests cluster within the same scheduling interval.

Stress Testing (High Concurrency Burst). A burst test was conducted with 1,000 requests fired simultaneously using 200 worker threads.

- Success rate: 100% (1,000/1,000)
- p50 latency: 3,479 ms
- p95 latency: 4,217 ms
- p99 latency: 4,324 ms
- Mean latency: 3,256 ms
- Maximum latency: 4,379 ms
- Throughput: 53.4 requests/second

Under a 1,000-way concurrent burst all requests were accepted without a single error. The elevated p50 of 3.5 s reflects journey-management planning load under extreme concurrency rather than Kafka overhead, which is eliminated by the singleton producer. A further mega-burst test of 2,000 simultaneous requests achieved 86.5 req/s with a 2.2 s mean latency and 100% success rate.

6.3.2 Cross-Region Booking Performance. Cross-region bookings were tested to evaluate end-to-end latency of distributed saga coordination across multiple regions.

- Success rate: 10/10 APPROVED
- Mean end-to-end latency: 401 ms
- p50 end-to-end latency: 407 ms
- p95 end-to-end latency: 468 ms
- Min / Max end-to-end: 340 ms / 468 ms
- Mean submission latency: 128 ms

These results show that a booking involving coordination across two independent validation services, two Redis instances, and multiple Kafka round-trips completes in under half a second on average. The tight spread of 128 ms between minimum and maximum end-to-end latency demonstrates consistent saga coordination. Variance is primarily due to Kafka consumer poll timing: when both regional outcomes arrive within the same poll cycle the saga resolves faster; when one arrives in the next cycle it adds approximately 100 ms.

6.3.3 Scalability and Load Distribution. The system scales well horizontally through Kafka consumer groups and stateless service replication. Under load, requests were successfully distributed across services without failure. The system maintained a 100% acceptance rate with no request drops, indicating that our architecture prevented overload failures.

6.3.4 Capacity Enforcement and Consistency. Capacity enforcement was tested under simultaneous contention by issuing 50 concurrent booking requests for a road segment with capacity set to 1.

- Approved bookings: 1
- Rejected bookings: 49
- Result: PASS
- Submission p50 latency: 594 ms
- Submission p95 latency: 815 ms
- Wall time: 866 ms

Exactly one booking was approved regardless of arrival order or timing, confirming that the Redis `SET NX` lock is atomic under concurrent access. The first Validation Service consumer to acquire the lock increments the counter from 0 to 1; all subsequent checks find the segment at capacity and reject immediately. The 594 ms p50 reflects the Kafka singleton producer improvement — the same test prior to hardening recorded a p50 of 3,203 ms under far fewer concurrent requests.

6.3.5 Reliability and Fault Tolerance. Fault tolerance was validated through controlled failure scenarios.

- **Asynchronous processing:** Booking requests return HTTP 202 immediately, with validation performed asynchronously through Kafka, ensuring that clients are never blocked waiting for distributed operations.
- **Container crash recovery:** When a Validation Service container fails, Docker restarts it automatically and Kafka replays uncommitted messages, ensuring no booking requests are lost.
- **Saga atomicity:** In cross-region bookings, partial failures are handled correctly. If one region rejects a sub-booking,

compensating transactions roll back previously approved segments, ensuring no partial state remains.

- **Successful saga coordination:** When all regions approve, the saga is committed and the booking is written consistently across regions, demonstrating correct distributed coordination.
- **Cancellation reliability:** Cross-region booking cancellation correctly updates the owning region and releases capacity across all involved regions.
- **Cross-region consistency:** All booking history queries use a scatter-gather pattern across regional data services, ensuring globally consistent views of booking data

6.3.6 Observability. System observability was verified through the health dashboard, which continuously polls all service `/health` endpoints across regions.

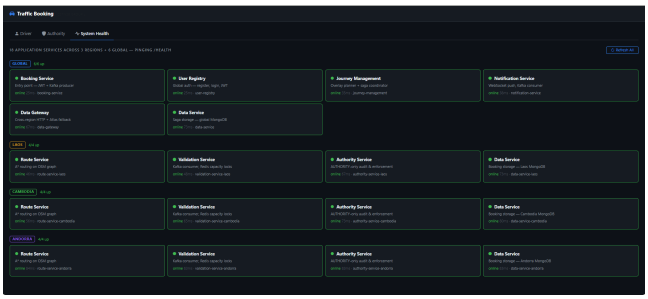

Figure 9: System health dashboard showing all services across three regions. All services are operational during testing.

The dashboard confirmed that all 18 services remained operational during testing, providing real-time visibility into system status and enabling rapid detection of failures.

6.3.7 Summary of Results. Overall, the system successfully met both functional and non-functional requirements. End-to-end testing confirmed that all core features, including booking submission, cross-region coordination, cancellation, and authority operations, behaved correctly under both success and failure conditions. Real-time WebSocket notifications functioned reliably, and booking data remained consistent across regions.

From a non-functional perspective, the system demonstrated strong performance, scalability, and reliability. Booking requests were processed with low latency under normal conditions, while cross-region coordination completed efficiently. Under higher load, latency increased but the system maintained a high success rate with no request loss, indicating graceful degradation. Distributed locking ensured strict capacity enforcement, and fault tolerance mechanisms such as container restarts and message replay prevented data loss. System observability was also achieved through health endpoints and a monitoring dashboard.

Future Testing Work. Further testing could be conducted to evaluate the system under more extreme or production-like conditions:

- Multi-region failure testing to simulate complete regional outages and verify DNS failover.
- Network partition testing to evaluate saga behaviour under high latency or packet loss.
- Long-running endurance tests to measure performance degradation over time.
- Horizontal scaling tests to measure performance improvements when replicating bottleneck services.
- Kafka failure testing to validate message durability under broker restarts or partition failures (Extending to multiple Kafka queues).

7 Allocation of Work

- **Kartik Singhal:** Journey Management Service and Route Service
- **Stevin Joseph Sebastian:** Booking Service and cancellation flow
- **Dylan Murray:** NGINX/Gateway, Docker regional clusters, and Authority Service
- **Raghav Gupta:** Validation Service, Flow diagrams
- **Niket Ghai:** Notification Service, User Registry, Architecture Diagram
- **Dylan Thompson:** MongoDB, Redis, Data Service layer, and report writing
- **Entire Team:** Frontend UI development
- **Entire Team:** System architecture design
- **Entire Team:** Testing and bug fixing

The Architecture was designed collaboratively by the team, we then split up the services and middle-ware implementations based on previous experience or interest expressed at weekly meetings after class. Some roles changed dynamically as architecture changed after the interim report feedback and as we learned more about the system through development.

8 Summary

Our project successfully implemented a globally-accessible distributed traffic booking system that meets both functional and non-functional requirements. The system supports booking, cancellation, and cross-region journeys, with real-time notifications through WebSockets.

Key achievements include the design of a multi-region microservices architecture using geographic partitioning, and the use of middleware components to enable communication, coordination, and data management across services. These include asynchronous messaging with Kafka, GeoDNS, API routing through NGINX, and data access layers. The system also implements the saga pattern to ensure consistency in cross-region bookings. Redis-based distributed locking enforces strict capacity constraints under scaling loads, while fault tolerance is achieved through container restarts and failover mechanisms, with MongoDB acting as the global source of truth.

In conclusion, our system demonstrates a practical application of distributed systems principles, achieving scalable, reliable, and consistent behaviour in a multi-region environment.